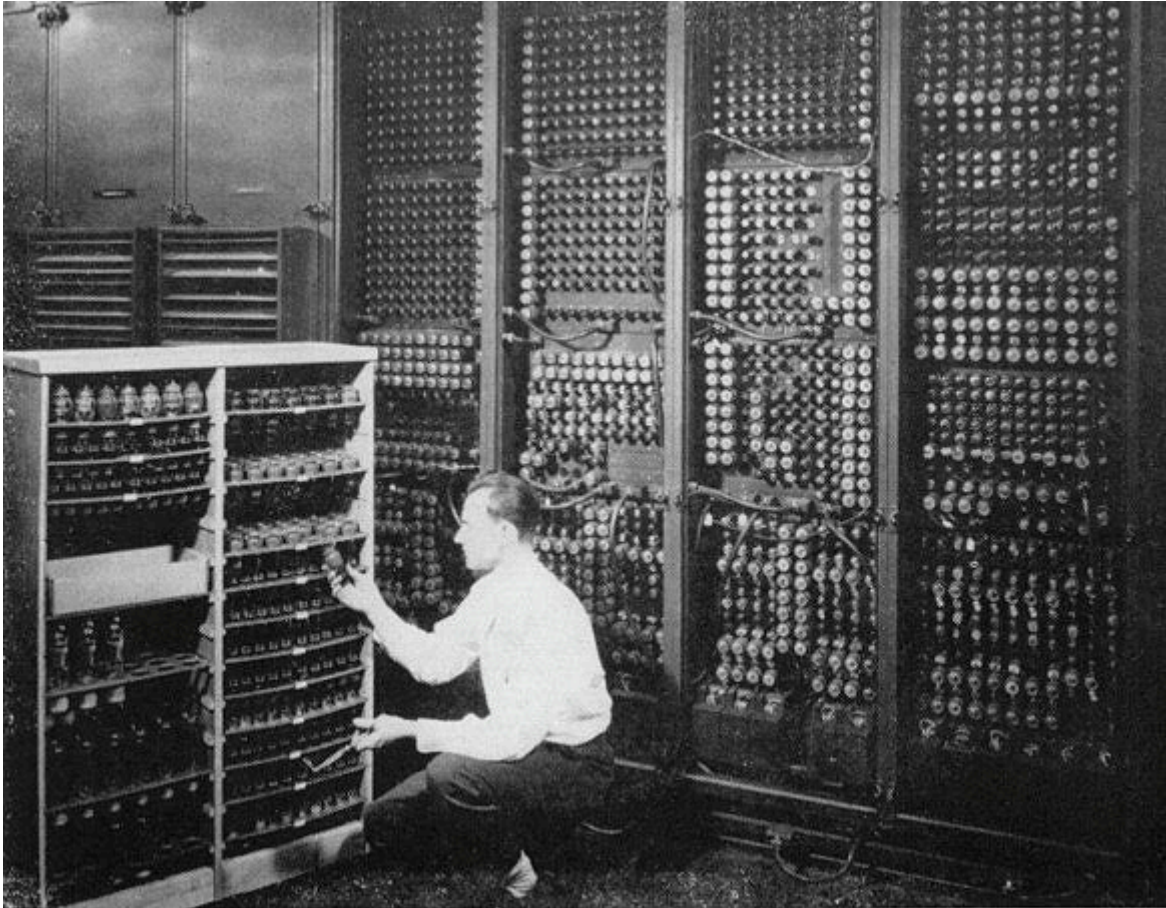


Práctica Final - Última Parte

Traductor frontend: Lenguaje C a LISP

Traductor backend: LISP a notación Postfix



Miguel Jimeno Casas - 100495932 - 100495932@alumnos.uc3m.es

Héctor Herraiz Díez - 100499734 - 100499734@alumnos.uc3m.es

Grupo 82 - Ingeniería Informática

1.Introducción

En este proyecto se parte de un traductor que realiza la conversión de un subconjunto del lenguaje C a expresiones en LISP, actuando como *frontend*. El objetivo planteado ha sido desarrollar el *backend* de la herramienta, de manera que, a partir del código intermedio generado en LISP, se produzca automáticamente el código equivalente en el lenguaje FORTH. Para ello, hemos utilizado Bison como generador de analizadores sintácticos, apoyándonos en una plantilla inicial ([trad1e.y](#)) que proporcionaba la base de la gramática LISP.

El trabajo ha consistido en adaptar esta plantilla para crear un nuevo fichero ([back4.y](#)) que implemente correctamente las especificaciones requeridas para la traducción LISP → FORTH. A lo largo del desarrollo, hemos tenido que diseñar la gramática adecuada, gestionar la generación directa de código FORTH y resolver las diferencias semánticas y sintácticas entre ambos lenguajes, especialmente en lo que respecta a la gestión de variables, las estructuras de control y la forma de evaluar expresiones.

Además, antes de abordar el backend, completamos el desarrollo del frontend en LISP, partiendo de [trad3.y](#) y finalizando en [trad4.y](#) para el trabajo final. En esta fase, realizamos las últimas modificaciones necesarias para cumplir con las especificaciones completas del proyecto, como la correcta gestión de variables locales dentro de funciones y el soporte para el manejo de vectores.

De cara a la próxima entrega, se prevé realizar una batería más amplia de pruebas que permita validar exhaustivamente el funcionamiento de ambos traductores —frontend y backend—, comprobando la correcta traducción de distintos programas desde el código fuente en C hasta su equivalente final en FORTH, pasando por su representación intermedia en LISP. Estas pruebas incluirán casos con funciones recursivas, estructuras de control complejas, manipulación de vectores y anidamiento de expresiones, con el objetivo de asegurar la robustez y completitud de la herramienta desarrollada.

2. Frontend por completar

En la fase final del desarrollo del frontend ([trad4.y](#)), se corrigió la gestión de variables locales dentro de funciones. Anteriormente, el manejo del [current_function](#) era incorrecto, lo que afectaba al tratamiento de las variables locales. Se modificó la regla de función en la gramática para actualizar

correctamente el `current_function` y así poder gestionar las variables locales de forma precisa. El cambio fue el siguiente:

C/C++

funcion:

```
IDENTIF '(' parametros ')' '{' instrucciones '}' {

    current_function = $1.code;

    // Verificar si la última instrucción es un return

    char *last_instr = strrchr($6.code, '\n');

    if (last_instr != NULL) last_instr++;

    else last_instr = $6.code;

    if (strstr(last_instr, "return") != NULL) {

        // Si termina con return, usamos el valor directamente

        sprintf(temp, "(defun %s (%s)\n%s)", $1.code, $3.code, $6.code);

    } else {

        // Si no termina con return, agregamos nil al final

        sprintf(temp, "(defun %s (%s)\n%s\n)", $1.code, $3.code,
            $6.code);

    }

    $$code = gen_code(temp);

    free(current_function);

}
```

;

Esta modificación fue esencial para soportar correctamente las variables locales tanto en la generación de código LISP como posteriormente en su traducción a FORTH.

Por último, en [trad4.y](#), también se implementó el soporte para el uso de vectores (arrays). Se añadió una nueva construcción para permitir asignaciones a posiciones concretas de un vector, diferenciando si el vector es una variable local o global:

```
C/C++
| IDENTIF '[' NUMBER ']' '=' expresion {
    if (is_local_var($1.code)) {
        sprintf(temp, "(setf (aref %s_%s %d) %s)", current_function,
$1.code, $3.value, $6.code);
    } else {
        sprintf(temp, "(setf (aref %s %d) %s)", $1.code, $3.value, $6.code);
    }
    $$code = gen_code(temp);
}
```

Además, se añadieron las reglas necesarias para permitir la inicialización y definición de vectores tanto locales como globales. Por último los añadimos para poder utilizar como operandos.

```
Unset
| IDENTIF '[' NUMBER ']' {
    if(current_function != NULL
    && is_local_var($1.code)) {
        // Variable local o
        parámetro
        sprintf(temp, "aref
        %s_%s %d", current_function, $1.code, $3.value);
    } else {
        // Variable global
        sprintf(temp, "aref %s
        %d", $1.code, $3.value);
    }
    $$code = gen_code(temp);
}
```

3. Desarrollo del Backend

El punto de partida para la implementación del backend ha sido el fichero `trad1e.y`, el cual contenía una gramática LISP básica y un esqueleto de traductor basado en Bison. Para convertirlo en un traductor funcional hacia FORTH, se ha creado el nuevo fichero `back4.y`, realizando las siguientes modificaciones y ampliaciones principales:

Adaptación de la gramática

Se han ajustado las reglas para reconocer correctamente las construcciones de LISP necesarias (funciones, variables, operaciones, estructuras de control) de acuerdo con las necesidades de la generación de código FORTH.

Traducción directa

Siguiendo la recomendación del enunciado, se ha optado por un enfoque de traducción directa en el que, a medida que se reconocen los nodos del árbol sintáctico, se emite inmediatamente el código FORTH correspondiente, evitando la construcción de árboles intermedios o la necesidad de un recorrido diferido.

Gestión de variables

Las variables globales se traducen utilizando la palabra clave `variable` en FORTH. Las asignaciones de valores a variables se implementan utilizando los operadores `!` y `@`, tal como se especifica.

Definición de funciones

Se traduce la definición de funciones genéricas mediante la sintaxis `: <nombre> ... ;` de FORTH, asegurando una correcta gestión de espacios, como es requerido en la convención del lenguaje.

Operaciones y operadores

Se ha establecido la correspondencia directa entre los operadores de LISP y su equivalente en FORTH. Por ejemplo, `and` y `or` se traducen directamente, mientras que operadores como `=` requieren emitir primero un `=` y luego un `0=` si es necesario ajustar la semántica.

Estructuras de control

- El bucle `while` en LISP se traduce en FORTH usando la estructura `begin ... while ... repeat`, asegurando la conversión postfija de la condición y el cuerpo.
- La estructura `if-else` se traduce utilizando `if ... else ... then`, y en el caso de no existir rama else, simplemente `if ... then`.

Impresión de datos

Se ha manejado la impresión de cadenas de texto mediante el uso de `." texto "` en FORTH, cuidando el espacio obligatorio tras `."`. La impresión de valores numéricos se realiza mediante el operador `..`.

Durante todo el desarrollo, se ha tenido especial cuidado en la correcta impresión de espacios entre los operadores y operandos, ya que FORTH es sensible a la separación por espacios para la correcta interpretación del código.

Gracias a este proceso, el backend implementado en `back4.y` permite transformar el código LISP generado por el frontend en un programa funcional en FORTH, respetando las convenciones del lenguaje y cumpliendo los requisitos especificados.